
boolean



Routing

Dove vado quando arriva una richiesta?

Express Js - Routing

Il termine ***Routing*** o instradamento è il processo di selezione del percorso che una **Richiesta HTTP** deve prendere.

In altre parole è l'operazione che permette di determinare come l'applicazione deve rispondere a seguito di una **Request** del client ad un determinato **URI** (o path) fatta con uno specifico metodo HTTP (GET, POST, ecc...)

Come tutti i framework, anche **Express** ha un sistema di routing integrato che facilita questa operazione.

Express Js - Routing

Tramite Express possiamo definire tutti gli indirizzi (detti **rotte)** di cui abbiamo bisogno per la nostra applicazione. Tramite l'oggetto **app**, per ogni rotta possiamo specificare 3 elementi:

1. **metodo HTTP** che dovrà essere usato per accedere alla rotta (per il momento *get*)
2. **path** ossia l'URL della rotta
3. **funzione** che ha lo scopo di rispondere quando viene richiamata la rotta

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', function (req, res) {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```

Express Js - Request e Response

Non dimentichiamo che la request e la response, vengono fornite automaticamente come parametri alla funzione che gestisce una rotta:

req: (request) oggetto che rappresenta la richiesta e contiene delle proprietà per accedere alle query string, parametri dinamici, dati ricevuti e molto altro.

res: (response) oggetto che rappresenta la risposta che l'applicazione Express invia a seguito di una richiesta. Permette di specificare i dati da inviare, la loro tipologia o altre operazioni.

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', function (req, res) {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```

Nel caso del routing ci interessa la request



Express Js - Parametri dinamici

Oltre a definire il path di una rotta, possiamo prevedere anche dei **parametri dinamici**.

Questi sono porzioni dell'URI che vengono riservate ed utilizzate per recuperare dei valori non statici, comunemente passati dal browser, come l'id di una risorsa.

Per definire un parametro dinamico, scegliamo una porzione di URI e ne assegniamo un **nome.**

Precediamo poi il nome con il simbolo **:**

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.get('/products/:id', function (req, res) {
10   // gestione rotta
11 })
12
13 app.listen(port, () => {
14   console.log(`App in ascolto sulla porta ${port}`)
15 })
```

Express Js - Parametri dinamici

All'interno della funzione, possiamo accedere a questo parametro proprio tramite l'oggeto **req.**

Più precisamente con

req.params.nome_parametro

Quando la rotta viene invocata, *al posto del parametro dinamico* occorre inserire *il valore che si vuole passare*:

`http://localhost:3000/products/45`

`http://localhost:3000/products/2`

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.get('/products/:id', function (req, res) {
10
11   console.log(req.params.id)
12 })
13
14 app.listen(port, () => {
15   console.log(`App in ascolto sulla porta ${port}`)
16 })
```

LIVE CODING

Proviamo a usare un parametro dinamico



CRUD e REST

Ovvero: come avere il controllo delle nostre risorse

Operazioni CRUD

Per avere il controllo totale della nostra applicazione, dobbiamo avere la capacità di manipolare i nostri dati, eseguendo una lista di operazioni di base, definite **C.R.U.D.**

CREATE => L'abilità di creare una risorsa (es: inserire una nuova pizza)

READ => L'abilità di accedere alle risorse (es: visualizzare la lista delle pizze o una singola pizza)

UPDATE => L'abilità di modificare una risorsa (es: cambiare il nome, la foto o gli ingredienti di una pizza)

DELETE => L'abilità di eliminare una risorsa (es: rimuovere una pizza dal menù)



Ci serve uno standard chiaro, semplice e inequivocabile per le operazioni CRUD

Questo standard è rappresentato dalle convenzioni REST

REST: REpresentational State Transfer

Un'architettura chiara, organizzata e condivisa

REST introduce il concetto di **risorse:** all'interno di ogni applicazione, abbiamo alcune **entità centrali** che giocano un ruolo fondamentale nel nostro progetto, che vengono definite **risorse**.

Per esempio:

- se creo un e-commerce, i **prodotti** sono una risorsa centrale.
- se creo un menù per la pizzeria, le **pizze** sono una risorsa centrale.
- se creo un blog, i **post** sono una risorsa.

Nell'e-commerce saranno risorse anche gli **utenti,** i **pagamenti** e tanto altro!



REST: REpresentational State Transfer

Un'architettura chiara, organizzata e condivisa

Nell'architettura REST le **risorse** sono in fin dei conti fonti di informazioni a cui si può accedere tramite un **identificatore globale** (un **URI**).

Per utilizzare le **risorse** , client e server comunicano attraverso una interfaccia standard (HTTP) e si scambiano **rappresentazioni** di queste risorse.

REST: REpresentational State Transfer

Come funziona?

Il funzionamento prevede:

1. Un **nome da assegnare alle rotte** per ciascuna operazione
2. una **struttura degli URL ben definita e univoca** per ogni risorsa (*più precisamente la parte finale, detta* ***endpoint*****)**
3. l'utilizzo dei **metodi HTTP** per azioni specifiche sulle risorse:
    - **GET** per il recupero dei dati
    - **POST** per la creazione di nuovi dati
    - **PUT/PATCH** per l'aggiornamento dei dati esistenti
    - **DELETE** per la cancellazione di dati

REST: REpresentational State Transfer

Come funziona?

Combinando i Verbi HTTP e gli **endpoint**, otteniamo queste combinazioni univoche, con le quali creeremo tutte le rotte necessarie alle nostre risorse**.**

Operazione sulla risorsa	**Nome Rotta**	**Verbo HTTP**	**Endpoint**
Read: Visualizzare tutti gli elementi	Index	GET	`pizzas/`
Read: Visualizzare un elemento	Show	GET	`pizzas/:id`
Create: Creare un nuovo elemento	Store	POST	`pizzas/`
Update: Modificare interamente un elemento	Update	PUT	`pizzas/:id`
Update: modificare parzialmente un elemento	Modify	PATCH	`pizzas/:id`
Delete: Eliminare un elemento	Destroy	DELETE	`pizzas/:id`

*Notiamo che se abbiamo **endpoint uguali,** abbiamo **verbi HTTP differenti**.*

*Notiamo che il **nome della risorsa nell'endpoint è sempre al plurale***

*Notiamo inoltre che **quando bisogna recuperare un singolo elemento**, abbiamo sempre un **parametro dinamico nell'endpoint**!*

REST: REpresentational State Transfer

Quindi, se vogliamo creare ad esempio le rotte per le nostre pizze, avremo tutte le rotte col nome della risorsa al plurale (**/pizzas)** e le rotte finali saranno:

[GET] **/pizzas** *(index)*

[GET] **/pizzas/:id** *(show)*

[POST] **/pizzas** *(store)*

[PUT] **/pizzas/:id** *(update)*

[PATCH] **/pizzas/:id** *(modify)*

[DELETE] **/pizzas/:id** *(destroy)*

```
1  // index
2  app.get('/pizzas', function (req, res) {
3    res.send('Lista delle pizze');
4  });
5  // show
6  app.get('/pizzas/:id', function (req, res) {
7    res.send('Dettagli della pizza ' + req.params.id);
8  });
9  // store
10 app.post('/pizzas', function (req, res) {
11   res.send('Creazione nuova pizza');
12 });
13
14 // update
15 app.put('/pizzas/:id', function (req, res) {
16   res.send('Modifica integrale della pizza ' + req.params.id);
17 });
18
19 // modify
20 app.patch('/pizzas/:id', function (req, res) {
21   res.send('Modifica parziale della pizza ' + req.params.id);
22 });
23
24 // destroy
25 app.delete('/pizzas/:id', function (req, res) {
26   res.send('Eliminazione della pizza ' + req.params.id);
27 });
```

LIVE CODING

Creiamo le rotte per le pizze!



Route Grouping

Mettiamo un po' d'ordine!

REST: REpresentational State Transfer

Un po' di organizzazione

Guardando il nostro file, ci rendiamo subito conto che tutte le rotte delle pizze, iniziano con /**pizzas**

Se avessimo altre risorse, ad esempio le bibite, avremmo poi tante rotte che iniziano con /**drinks**



Ma se l'URI di più rotte è lo stesso, non sarebbe meglio raggrupparle?

```
1  // index
2  app.get('/pizzas', function (req, res) {
3    res.send('Lista delle pizze');
4  });
5  // show
6  app.get('/pizzas/:id', function (req, res) {
7    res.send('Dettagli della pizza ' + req.params.id);
8  });
9  // store
10 app.post('/pizzas', function (req, res) {
11   res.send('Creazione nuova pizza');
12 });
13
14 // update
15 app.put('/pizzas/:id', function (req, res) {
16   res.send('Modifica integrale della pizza ' + req.params.id);
17 });
18
19 // modify
20 app.patch('/pizzas/:id', function (req, res) {
21   res.send('Modifica parziale della pizza ' + req.params.id);
22 });
23
24 // destroy
25 app.delete('/pizzas/:id', function (req, res) {
26   res.send('Eliminazione della pizza ' + req.params.id);
27 });
```

Express Js

Certo che sì grazie al...



Express Js

Certo che sì grazie al...

Router



Express Js - Router

Tramite la classe **express.Router** possiamo creare un *file dedicato* ad un gruppo di rotte

In questo file, andremo ad indicare tutte le rotte che riguardano una determinata entità, lasciando più asciutto **app.js** e organizzando meglio il nostro codice:

Da quel momento in poi, sappiamo che tutte le rotte delle pizze sono nel loro bel file dedicato!

Express Js - Router

Tramite la classe **express.Router** possiamo creare un *file dedicato* ad un gruppo di rotte

In questo file, andremo ad indicare tutte le rotte che riguardano una determinata entità, lasciando più asciutto **app.js** e organizzando meglio il nostro codice:

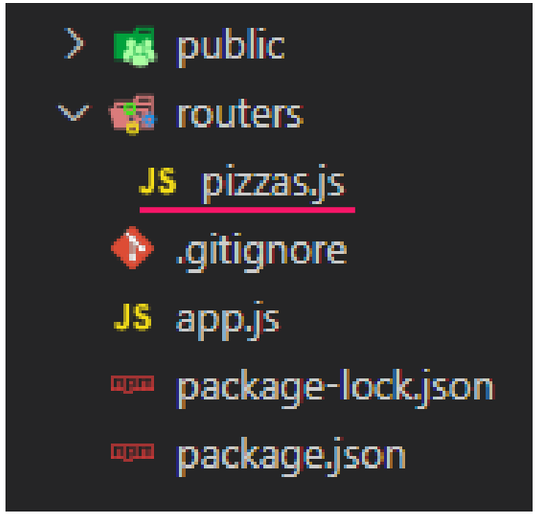
Da quel momento in poi, sappiamo che tutte le rotte delle pizze sono nel loro bel file dedicato!



Express Js - Router

Come si crea questo router?

Creiamo una cartella **routers** all'interno della quale aggiungeremo un **file per ogni entità** di cui vogliamo raggruppare le rotte.



Per tutte le rotte relative alle **pizze** creiamo un file chiamato **pizzas.js**

Invece per tutte le rotte relative all'entità **user** creiamo un file chiamato **users.js**

Express Js - Router

All'interno di ogni file dentro la cartella routers:

1. importiamo **Express**
2. creiamo una variabile **router** il cui valore sarà un istanza di express.Router()
3. invece di **`app`**, utilizziamo **`router`** per definire le rotte
4. Rimuoviamo il nome della risorsa dagli URI (**`/pizzas`**)
5. esportiamo l'istanza di router

```
1  const express = require('express')
2  const router = express.Router();
3
4  // index
3  router.get('/', function(req, res) {
6    res.send('Lista delle pizze');
7  });
8
9  // show
3  router.get('/:id', function(req, res) {
11   res.send('Dettagli della pizza ' + req.params.id);
12 });
13
14 // store
3  router.post('/', function(req, res) {
16   res.send('Creazione nuova pizza');
17 });
18
5  module.exports = router;
```

Express Js - Router

Tutto qui?

No, dobbiamo comunicare all'app principale l'esistenza di queste rotte.

1. **Importiamo il router** appena creato
2. Tramite la funzione **use** indichiamo a express che esistono nuove rotte, specificando:
   - Quale deve essere il prefisso (**`"/pizzas"`**)
   - Quale router utilizzare

```
1  const express = require("express");
2  const app = express();
3  const port = 3000;
1  const pizzasRouter = require('./routers/pizzas');
5
6  app.use(express.static('public'));
7
8  app.get("/", (req, res) => {
9    res.send(`Hello world`);
10 });
11
2  app.use("/pizzas", pizzasRouter)
13
14 app.listen(port, () => {
15   console.log(`App attiva su
   http://localhost:${port}`);
16 });
```

Express Js - Router

Ricapitolando, la funzione **use** si aspetta 2 argomenti

- **path** base delle rotte. Questo verrà aggiunto come prefisso ad ogni rotta creata tramite il router
- **istanza del router**

```
1  const express = require("express");
2  const app = express();
3  const port = 3000;
4  const pizzasRouter =
   require('./routers/pizzas');
5
6  app.use(express.static('public'));
7
8  app.get("/", (req, res) => {
9    res.send(`Hello world`);
10 });
11
12 app.use("/pizzas", pizzasRouter)
13
14 app.listen(port, () => {
15   console.log(`App attiva su
   http://localhost:${port}`);
16 });
```

Express Js - Routers

Se in **routers/pizzas.js** abbiamo creato le rotte index, show e store
e in **app.js** le abbiamo registrate con il prefisso **/pizzas**, le rotte finali saranno:

[GET] **/** *(index)* ----------→ [GET] http://localhost:3000**/pizzas/** *(index)*

[GET] **/:id** *(show)* ----------→ [GET] http://localhost:3000**/pizzas/:id** *(show)*

[POST] **/** *(store)* ----------→ [POST] http://localhost:3000**/pizzas/** *(store)*

routers/pizzas.js

```
1  // index
2  router.get('/', function(req, res) {
3    res.send('Lista delle pizze');
4  });
5  // show
6  router.get('/:id', function(req, res) {
7    res.send('Dettagli della pizza ' + req.params.id);
8  });
9  // store
10 router.post('/', function(req, res) {
11   res.send('Creazione nuova pizza');
12 });
```

app.js

```
1  // ...
2  //
3  const pizzasRouter = require('./routers/pizzas');
4
5  // ...
6
7  app.use("/pizzas", pizzasRouter)
8
9  // ...
```

LIVE CODING

Creiamo il router per le pizze!



ESERCITAZIONE